It tends to be easier to visualize W^TW than W.

Here we see that W^TW is an **identity matrix** for the most important features and **0** for less important ones.

We can also look at the bias, b.

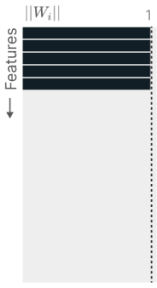
The bias is **zero** for features learned to pass though, and the **expected value** (a positive number) for others.

Weight / Bias Element Values

But the thing we really care about is this hypothesized phenomenon of superposition – does the model represent "extra features" by storing them non-orthogonally? Is there a way to get at it more explicitly? Well, one question is just how many features the model learns to represent. For any feature, whether or not it is represented is determined by $||W_i||$, the norm of its embedding vector.

We'd also like to understand whether a given feature shares its dimension with other features. For this, we calculate $\sum_{j \neq i} (\hat{W}_i \cdot W_j)^2$, projecting all other features onto the direction vector of W_i. It will be 0 if the feature is orthogonal to other features (dark blue below). On the other hand, values ≥ 1 mean that there is some group of other features which can activate W_i as strongly as feature i itself!

We can visualize the model we looked at previously this way:



We want to understand which features the model chooses to represent in its hidden representation, and whether they're orthogonal to each other.

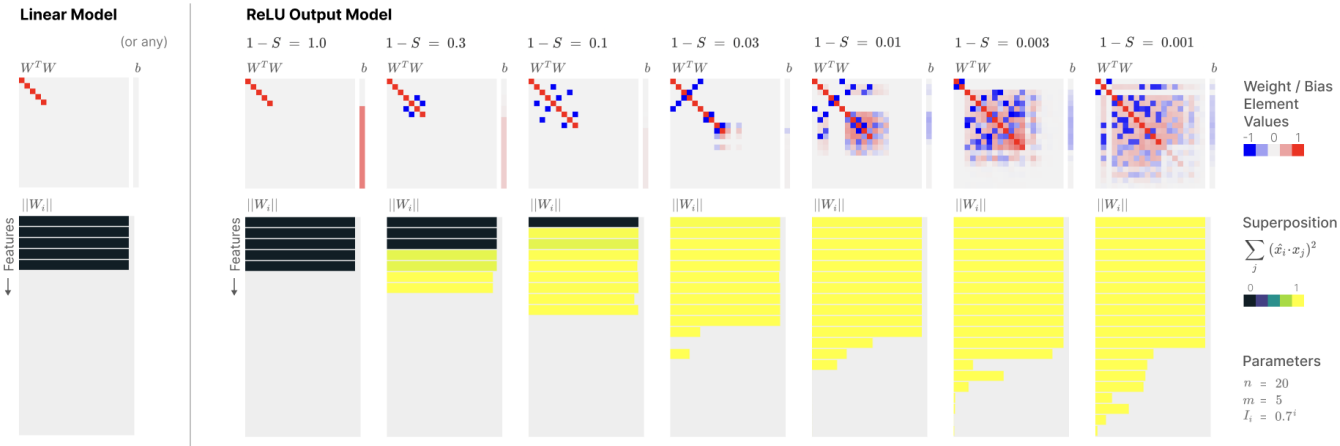
To do this, we visualize the norm of each feature's direction vector, $||W_i||$. This will be ~1 if a feature is fully represented, and zero if it is not. For each feature, we also use color to visualize whether it is orthogonal to other features (i.e. in superposition).

This model simply dedicates one dimension to each of the most important features, representing them orthogonally.

Superposition

$\sum_j (\hat{x}_i \cdot x_j)^2$

Now that we have a way to visualize models, we can start to actually do experiments. We'll start by considering models with only a few features ($n = 20;\ m = 5;\ I_i = 0.7^i$). This will make it easy to visually see what happens. We consider a linear model, and several ReLU-output models trained on data with different feature sparsity levels:



Linear models learn the top m features. $1-S$ = 0.001 is shown, but others are similar.

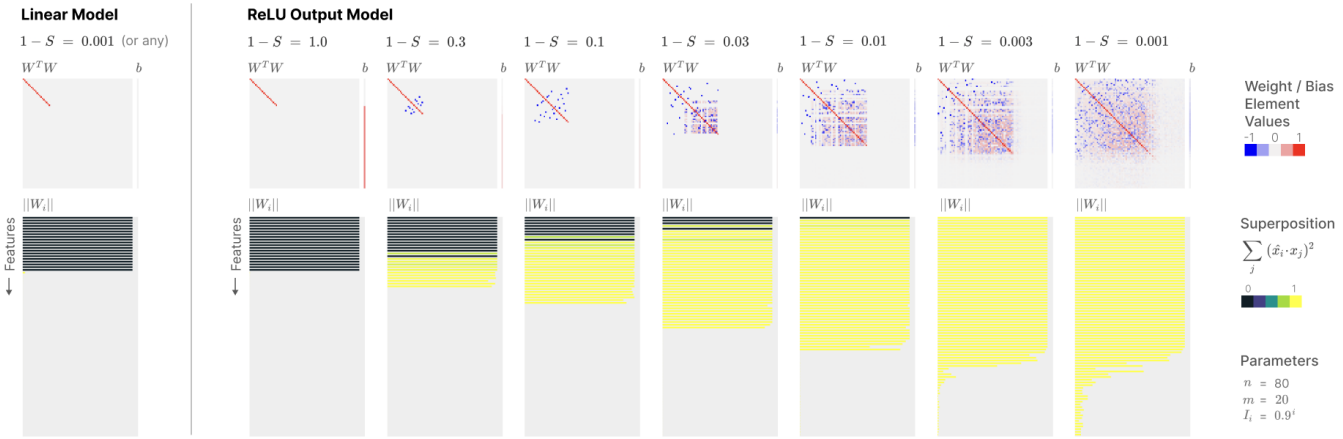
In the **dense** regime, ReLU output models also learn the top m features.

As **sparsity increases**, superposition allows models to represent more features. The most important features are initially untouched. This early superposition is organized in antipodal pairs (more on this later).

In the **high sparsity** regime, models put all features in superposition, and continue packing more. Note that at this point we begin to see positive interference and negative biases. We'll talk about this more later.

As our standard intuitions would expect, the linear model always learns the top-m most important features, analogous to learning the top principal components. The ReLU output model behaves the same on dense features ($1 - S = 1.0$), but as sparsity increases, we see superposition emerge. *The model represents more features by having them not be orthogonal to each other.* It starts with less important features, and gradually affects the most important ones. Initially this involves arranging them in antipodal pairs, where one feature's representation vector is exactly the negative of the other's, but we observe it gradually transition to other geometric structures as it represents more features. We'll discuss feature geometry further in the later section, The Geometry of Superposition.

The results are qualitatively similar for models with more features and hidden dimensions. For example, if we consider a model with $m = 20$ hidden dimensions and $n = 80$ features (with importance increased to $I_i = 0.9^i$ to account for having more features), we observe essentially a rescaled version of the visualization above:



Mathematical Understanding

In the previous section, we observed a surprising empirical result: adding a ReLU to the output of our model allowed a radically different solution – *superposition* – which doesn't occur in linear models.

The model where it occurs is still quite mathematically simple. Can we analytically understand why superposition is occurring? And for that matter, why does adding a single non-linearity make things so different from the linear model case? It turns out that we can get a fairly satisfying answer, revealing that our model is governed by balancing two competing forces – *feature benefit* and *interference* – which will be useful intuition going forwards. We'll also discover a connection to the famous Thomson Problem in chemistry.

Let's start with the linear case. This is well understood by prior work! If one wants to understand why linear models don't exhibit superposition, the easy answer is to observe that linear models essentially perform PCA. But this isn't fully satisfying: if we set aside all our knowledge and intuition about linear functions for a moment, why exactly is it that superposition can't occur?

A deeper understanding can come from the results of Saxe et al. [26] who study the learning dynamics of *linear neural networks* – that is, neural networks without activation functions. Such models are ultimately linear functions, but because they are the composition of multiple linear functions the dynamics are potentially quite complex. The punchline of their paper reveals that neural network weights can be thought of as optimizing a simple closed-form solution. We can tweak their problem to be a bit more similar to our linear case,[10] revealing the following equation:

$$L \sim \sum_i I_i(1-||W_i||^2)^2 + \sum_{i \neq j} I_j(W_j \cdot W_i)^2$$

Feature benefit is the value a model attains from representing a feature. In a real neural network, this would be analagous to the potential of a feature to improve predictions if represented accurately.

Interference betwen x_i and x_j occurs when two features are embedded non-orthogonally and, as a result, affect each other's predictions. This prevents superposition in linear models.

The Saxe results reveal that there are fundamentally two competing forces which control learning dynamics in the considered model. Firstly, the model can attain a better loss by representing more features (we've labeled this "feature benefit"). But it also gets a worse loss if it represents more than it can fit orthogonally due to "interference" between features.[11] In fact, this makes it never worthwhile for the linear model to represent more features than it has dimensions.[12]

Can we achieve a similar kind of understanding for the ReLU output model? Concretely, we'd like to understand $L = \int_x ||I(x - \mathrm{ReLU}(W^T W x + b))||^2 d\mathbf{p}(x)$ where x is distributed such that $x_i = 0$ with probability S.

The integral over x decomposes into a term for each sparsity pattern according to the binomial expansion of $((1-S)+S)^n$. We can group terms of the sparsity together, rewriting the loss as $L = (1-S)^n L_n + \ldots + (1-S)S^{n-1}L_1 + S^n L_0$, with each L_k corresponding to the loss when the input is a k-sparse vector. Note that as $S \to 1$, L_1 and L_0 dominate. The L_0 term, corresponding to the loss on a zero vector, is just a penalty on positive biases, $\sum_i \mathrm{ReLU}(b_i)^2$. So the interesting term is L_1, the loss on 1-sparse vectors:

$$L_1 = \sum_i \int_{0 \leq x_i \leq 1} I_i(x_i - \mathrm{ReLU}(||W_i||^2 x_i + b_i))^2 + \sum_{i \neq j} \int_{0 \leq x_i \leq 1} I_j \mathrm{ReLU}(W_j \cdot W_i x_i + b_j)^2$$

If we focus on the case $x_i = 1$, we get something which looks even more analagous to the linear case:

$$= \sum_i I_i(1 - \mathrm{ReLU}(||W_i||^2 + b_i))^2 + \sum_{i \neq j} I_j \mathrm{ReLU}(W_j \cdot W_i + b_j)^2$$

Feature benefit is similar to before. Note that ReLU never makes things worse, and that the bias can help when the model doesn't represent a feature by taking on the expected value.

Interference is similar to before but ReLU means that negative interference, or interference where a negative bias pushes it below zero, is "free" in the 1-sparse case.

This new equation is vaguely similar to the famous Thomson problem in chemistry. In particular, if we assume uniform importance and that there are a fixed number of features with $||W_i|| = 1$ and the rest have $||W_i|| = 0$, and that $b_i = 0$, then the feature benefit term is constant and the interference term becomes a generalized Thomson problem – we're just packing points on the surface of the sphere with a slightly unusual energy function. (We'll see this can be a productive analogy when we resume our empirical investigation in the following sections!)

Another interesting property is that ReLU makes negative interference free in the 1-sparse case. This explains why the solutions we've seen prefer to only have negative interference when possible. Further, using a negative bias can convert small positive interferences into essentially being negative interferences.

What about the terms corresponding to less sparse vectors? We leave explicitly writing these out to the reader, but the main idea is that there are multiple compounding interferences, and the "active features" can experience interference. In a later section, we'll see that features often organize themselves into sparse interference graphs such that only a small number of features interfere with another feature – it's interesting to note that this reduces the probability of compounding interference and makes the 1-sparse loss term more important relative to others.

Superposition as a Phase Change

The results in the previous section seem to suggest that there are three outcomes for a feature when we train a model: (1) the feature may simply not be learned; (2) the feature may be learned, and represented in superposition; or (3) the model may represent a feature with a dedicated dimension. The transitions between these three outcomes seem sharp. Possibly, there's some kind of phase change.[13]

One way to understand this better is to explore if there's something like a "phase diagram" from physics, which could help us understand when a feature is expected to be in one of these regimes. Although we can see hints of this in our previous experiment, it's hard to really isolate what's going on because many features are changing at once and there may be interaction effects. As a result, we set up the following experiment to better isolate the effects.

As an initial experiment, we consider models with 2 features but only 1 hidden layer dimension. We still consider the ReLU output model, $\mathrm{ReLU}(W^T W x - b)$. The first feature has an importance of 1.0. On one axis, we vary the importance of the 2nd "extra" feature from 0.1 to 10. On the other axis, we vary the sparsity of all features from 1.0 to 0.01. We then plot whether the 2nd "extra" feature is not learned, learned in superposition, or learned and represented orthogonally. To reduce noise, we train ten models for each point and average over the results, discarding the model with the highest loss.

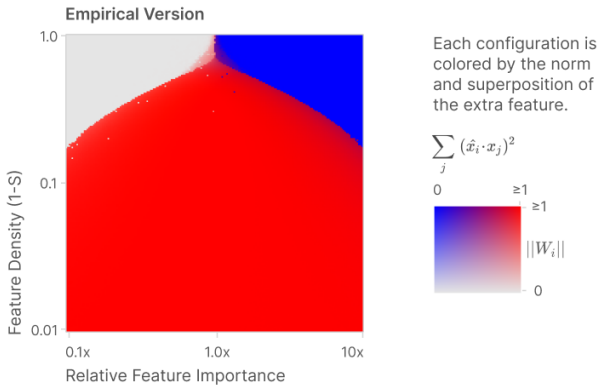
We can compare this to a theoretical "toy model of the toy model" where we can get closed form solutions for the loss of different weight configurations as a function of importance and sparsity. There are three natural ways to store 2 features in 1 dimension: $W = [1, 0]$ (ignore $[0, 1]$, throwing away the extra feature), $W = [0, 1]$ (ignore $[1, 0]$, throwing away the first feature to give the extra feature a dedicated dimension), and $W = [1, -1]$ (store the features in superposition, losing the ability to represent $[1, 1]$, the combination of both features at the same time). We call this last solution “antipodal” because the two basis vectors $[1, 0]$ and $[0, 1]$ are mapped in opposite directions. It turns out we can analytically determine the loss for these solutions (details can be found in this notebook).

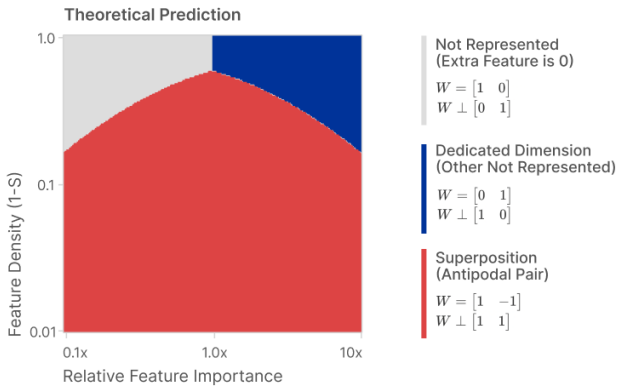
Sparsity-Relative Importance Phase Diagram (n=2, m=1)

What happens to an "extra feature" if the model can't give each feature a dimension? There are three possibilities, depending on feature sparsity and the extra feature's importance relative to other features:

- Extra Feature is Not Represented
- Extra Feature Gets Dedicated Dimension
- Extra Feature is Stored In Superposition

We can both study this empirically and build a theoretical model:





As expected, sparsity is necessary for superposition to occur, but we can see that it interacts in an interesting way with relative feature importance. But most interestingly, there appears to be a real phase change, observed in both the empirical and theoretical diagrams! The optimal weight configuration discontinuously changes in magnitude and superposition. (In the theoretical model, we can analytically confirm that there's a first-order phase change: there's crossover between the functions, causing a discontinuity in the derivative of the optimal loss.)

We can ask this same question of embedding three features in two dimensions. This problem still has a single "extra feature" (now the third one) we can study, asking what happens as we vary its importance relative to the other two and change sparsity.

For the theoretical model, we now consider four natural solutions. We can describe solutions by asking "what feature direction did W ignore?" For example, W might just not represent the extra feature – we'll write this $W \perp [0, 0, 1]$. Or W might ignore one of the other features, $W \perp [1, 0, 0]$. But the interesting thing is that there are two ways to use superposition to make antipodal pairs. We can put the "extra feature" in an antipodal pair with one of the others ($W \perp [0, 1, 1]$) or put the other two features in superposition and give the extra feature a dedicated dimension ($W \perp [1, 1, 0]$). Details on the closed form losses for these solutions can be found in this notebook. We do not consider a last solution of putting all the features in joint superposition, $W \perp [1, 1, 1]$.